claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-af85bdfcb57fc428d.jsonl`
- SHA-256: `1e555805e0e9a2fc5f5232b8d8c357b97c4bcdc1b1414192774d582f319617aa`
- Source modified: `2026-06-10T16:24:49+00:00`
- Imported at: `2026-07-05T16:48:21+00:00`
- project: `wf_27df53c8-8e0`
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`

Transcript

1. user (2026-06-10T16:22:08.053Z)

You are an adversarial verifier. A code reviewer audited the repo at C:/Users/avidu/Projects/muneem and reported this finding:

TITLE: Transient keychain read failure mints a new key and silently overwrites the only DB key
SEVERITY CLAIMED: high
FILE: src/muneem/db_key.py (line 24-31)
DESCRIPTION: `secrets.get_secret` (src/muneem/secrets.py:25-30) swallows every `KeyringError` and returns `None`, deliberately treating 'backend unavailable' the same as 'key absent'. `get_or_create_key()` then interprets that `None` as 'no key has ever been minted', generates a fresh key, and `set_secret` OVERWRITES the existing 'db-key' entry. If a read fails transiently (locked/temporarily failing Credential Manager session, keyring backend hiccup) while the subsequent write succeeds, the only copy of the real ledger key is destroyed; the next `connect_encrypted` fails with 'wrong key' and the data is gone unless a recovery file exists. `connect_encrypted` (src/muneem/db.py:207) calls `get_or_create_key()` unconditionally ? even when an encrypted ledger file already exists on disk, which is exactly the case where minting is never the right answer.
EVIDENCE QUOTED: def get_or_create_key() -> str:
    existing = _keychain.get_secret(_DB_KEY_NAME)
    if existing:
        return existing
    key = _stdlib_secrets.token_bytes(32).hex()
    _keychain.set_secret(_DB_KEY_NAME, key)
    return key
SUGGESTED FIX: Distinguish 'absent' from 'unavailable': add a get_secret variant that raises (or returns a sentinel) on KeyringError, and have get_or_create_key abort loudly instead of minting when the backend errored. Additionally, gate minting on the ledger file: if the encrypted DB file already exists but the keychain has no key, refuse with a pointer to 'muneem db recover' rather than minting a key that cannot open it.

Your job is to try to REFUTE it. Your lens: correctness: trace the actual code path the finding describes and check the claimed behavior is what the code really does (read the file and its callers/tests).
Read the actual code in the repo (and tests/config where relevant). If after reading you cannot confirm the problem is real, set isReal=false. If it is real but over/under-stated, set adjustedSeverity accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):
- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/, testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file existence and git-tracking status (git ls-files, git log --all -- <path>).
- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.

- Never include sensitive values in your findings. Refer to them abstractly ("a real account number", "a client_secret value").
- Do not use WebSearch/WebFetch and do not send any repo content to any external service. Structured output only.

2. user (2026-06-10T16:22:12.959Z)

```
1       """The ledger's encryption key ? minted once, kept in the OS keychain.
2
3       muneem's local ledger is **encrypted at rest** with SQLCipher (DESIGN.md §5.4 rule 4).
4       The 256-bit key is generated on first use and stored in the OS keychain via
5       :mod:`muneem.secrets` (Windows Credential Manager / macOS Keychain / Secret Service) ?
6       never on disk in plaintext, never in the repo, never logged.
7
8       It is handed to SQLCipher in **raw-key form** (``PRAGMA key = "x'<64 hex>'"``), which
uses
9       the 256-bit key directly and *skips* the passphrase KDF. That is correct here because
the
10      key is already full-entropy random; KDF stretching only buys security for low-entropy
human
11      passphrases (an opt-in we may add later ? see ROADMAP D.2 phased plan).
12      """
13
14      from __future__ import annotations
15
16      import secrets as _stdlib_secrets
17
18      from muneem import secrets as _keychain
19
20      # Keychain entry name (under the shared "muneem" service) holding the 64-hex-char DB
key.
21      _DB_KEY_NAME = "db-key"
22
23
24      def get_or_create_key() -> str:
25          """Return the ledger's 256-bit key as 64 hex chars, minting+storing one on first
use."""
26          existing = _keychain.get_secret(_DB_KEY_NAME)
27          if existing:
28              return existing
29          key = _stdlib_secrets.token_bytes(32).hex()
30          _keychain.set_secret(_DB_KEY_NAME, key)
31          return key
32
33
34      def peek_key() -> str | None:
35          """Return the stored key, or ``None`` if none has been minted yet. Never creates
one."""
36          return _keychain.get_secret(_DB_KEY_NAME)
37
38
39      def new_key() -> str:
40          """Generate a fresh random 256-bit key as 64 hex chars (not stored)."""
41          return _stdlib_secrets.token_bytes(32).hex()
42
43
44      def set_key(key: str) -> None:
45          """Overwrite the stored DB key ? used by key rotation, **after** the file has been
rekeyed."""
46          _keychain.set_secret(_DB_KEY_NAME, key)
47
```

3. user (2026-06-10T16:22:13.624Z)

```
1       """Secret storage via the OS keychain.
```

```
2
3       OAuth tokens, cached statement passwords, and the DB encryption key are stored here
(Windows
4       Credential Manager / macOS Keychain / Secret Service), never in the repo, on disk in
plaintext, or
5       in logs. See DESIGN.md section 5.
6
7       External-call discipline (every keyring call is wrapped with attribution):
8       - **read** (:func:`get_secret`) is *graceful* ? a missing or unavailable backend returns
``None``
9         (absence is a valid state; this also keeps headless CI happy).
10      - **write** (:func:`set_secret`) is *loud* ? a failure raises
:class:`~muneem.errors.KeychainError`,
11        because silently losing a write (e.g. the DB key) would be dangerous. The secret
*name* is
12        included for diagnosis; the value never is.
13      """
14
15      from __future__ import annotations
16
17      import keyring
18      from keyring.errors import KeyringError, PasswordDeleteError
19
20      from muneem.errors import KeychainError
21
22      _SERVICE = "muneem"
23
24
25      def get_secret(name: str) -> str | None:
26          """Return the stored secret, or ``None`` if absent or the keychain is
unavailable."""
27          try:
28              return keyring.get_password(_SERVICE, name)
29          except KeyringError:
30              return None
31
32
33      def set_secret(name: str, value: str) -> None:
34          """Store (or replace) a secret in the OS keychain. Raises KeychainError on
failure."""
35          try:
36              keyring.set_password(_SERVICE, name, value)
37          except KeyringError as exc:
38              # Name only ? never the value ? in the message (it may be a password or the DB
key).
39              raise KeychainError(f"could not store secret {name!r} in the OS keychain") from
exc
40
41
42      def delete_secret(name: str) -> None:
43          """Delete a secret if present; a no-op if it is not (or the keychain is
unavailable)."""
44          try:
45              keyring.delete_password(_SERVICE, name)
46          except (PasswordDeleteError, KeyringError):
47              pass
48
```

4. user (2026-06-10T16:22:19.479Z)

```
170         ``:memory:`` and tests. A keyed open never silently falls back to plaintext (see
171         :func:`_open_encrypted`); the production ledger is opened via
:func:`connect_encrypted`.
172         """
173         path = str(db_path)
```

```
        if path != ":memory:":
            Path(path).parent.mkdir(parents=True, exist_ok=True)
        if key is not None:
            conn = _open_encrypted(path, key)
            _harden_permissions(path)
        else:
            conn = sqlite3.connect(path)
            conn.row_factory = sqlite3.Row
        conn.execute("PRAGMA foreign_keys = ON")
        try:
            yield conn
            conn.commit()
        finally:
            conn.close()


    @contextmanager
    def connect_encrypted(db_path: Path | str) -> Iterator[sqlite3.Connection]:
        """Open the real on-disk ledger, encrypted with the keychain-managed key.

        The production path: resolves (or mints, on first use) the 256-bit key from the OS
        keychain and opens with SQLCipher. If ``db_path`` is a pre-existing *plaintext*
SQLite
        file, refuses with a clear pointer to ``muneem db encrypt`` rather than a cryptic
        "file is not a database" error.
        """
        from muneem.db_key import get_or_create_key

        path = str(db_path)
        if is_plaintext_sqlite(path):
            raise EncryptedDatabaseError(
                f"{Path(path).name} is an unencrypted SQLite database; run 'muneem db
encrypt' "
                "to migrate it to an encrypted ledger"
            )
        with connect(path, key=get_or_create_key()) as conn:
            yield conn


    def encrypt_database(db_path: Path | str, key: str) -> int:
        """Migrate a plaintext SQLite ledger to an encrypted one in place, via
``sqlcipher_export``.

        The original is preserved as ``<name>.plaintext.bak`` for the caller to verify and
then
        securely delete (it still holds the cleartext data). Returns the number of user
tables
        copied. Raises :class:`EncryptedDatabaseError` if ``db_path`` is not a plaintext
SQLite file.
        """
        import sqlcipher3

        path = Path(db_path)
        if not is_plaintext_sqlite(path):
            raise EncryptedDatabaseError(
                f"{path.name} is not a plaintext SQLite database (already encrypted or
missing)"
            )

        enc_tmp = path.with_name(path.name + ".enc.tmp")
        enc_tmp.unlink(missing_ok=True)
        try:
            src = sqlcipher3.connect(str(path))  # open the plaintext source (no key)
            try:
                src_tables = src.execute(
```

```
232                     "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
233                 ).fetchone()[0]
234                 target = str(enc_tmp).replace("\\", "/")  # SQL string literal: forward
slashes
235                 src.execute(f"ATTACH DATABASE '{target}' AS encrypted KEY \"x'{key}'\"")
236                 src.execute("SELECT sqlcipher_export('encrypted')")
237                 src.execute("DETACH DATABASE encrypted")
238             finally:
239                 src.close()
240
241             # Verify the encrypted copy BEFORE the destructive swap: it must open with the
key, pass
242             # an integrity check, and carry every table across. (The plaintext .bak is kept
either way,
243             # so a failed check is recoverable ? but we refuse to swap in a bad file.)
244             with connect(str(enc_tmp), key=key) as check:
245                 integrity = check.execute("PRAGMA integrity_check").fetchone()[0]
246                 ntables = check.execute(
247                     "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
248                 ).fetchone()[0]
249             if integrity != "ok" or ntables != src_tables:
```

5. user (2026-06-10T16:22:20.311Z)

```
src\muneem\cli.py:370:    from muneem.db_key import peek_key
src\muneem\cli.py:373:    key_state = "present in keychain" if peek_key() else "not yet minted"
src\muneem\cli.py:393:    from muneem.db_key import get_or_create_key
src\muneem\cli.py:406:    n = dbmod.encrypt_database(db_path, get_or_create_key())
src\muneem\cli.py:416:    from muneem.db_key import peek_key
src\muneem\cli.py:421:    key = peek_key()
src\muneem\cli.py:434:    from muneem import db_key
src\muneem\cli.py:438:    fresh = db_key.new_key()
src\muneem\cli.py:440:    db_key.set_key(fresh)
src\muneem\cli.py:444:        db_key.set_key(current)  # roll back; the file is still under the
old key
src\muneem\cli.py:482:    from muneem.db_key import peek_key
src\muneem\cli.py:484:    key = peek_key()
src\muneem\cli.py:514:    from muneem import db_key, recovery
src\muneem\cli.py:531:    db_key.set_key(key)
src\muneem\db.py:199:    from muneem.db_key import get_or_create_key
src\muneem\db.py:207:    with connect(path, key=get_or_create_key()) as conn:
src\muneem\db_key.py:24:def get_or_create_key() -> str:
src\muneem\db_key.py:34:def peek_key() -> str | None:
tests\test_db_crypto.py:94:def test_get_or_create_key_is_idempotent_and_64_hex(monkeypatch):
tests\test_db_crypto.py:96:    from muneem import db_key
tests\test_db_crypto.py:98:    assert db_key.peek_key() is None
tests\test_db_crypto.py:99:    k1 = db_key.get_or_create_key()
tests\test_db_crypto.py:101:    k2 = db_key.get_or_create_key()
tests\test_db_crypto.py:103:    assert db_key.peek_key() == k1
src\muneem\recovery.py:59:def wrap_key(db_key: str, passphrase: str) -> dict[str, Any]:
src\muneem\recovery.py:60:    """Return a JSON-serialisable recovery payload wrapping ``db_key``
under ``passphrase``."""
src\muneem\recovery.py:66:    ciphertext = AESGCM(key).encrypt(nonce, db_key.encode("utf-8"),
_AAD)
src\muneem\recovery.py:104:def write_recovery_file(path: Path | str, db_key: str, passphrase:
str) -> Path:
src\muneem\recovery.py:105:    """Wrap ``db_key`` under ``passphrase`` and write the recovery
file (owner-only perms)."""
src\muneem\recovery.py:106:    payload = wrap_key(db_key, passphrase)
tests\test_db_crypto_ops.py:16:from muneem import db_key  # noqa: E402
tests\test_db_crypto_ops.py:130:    old = db_key.peek_key()
tests\test_db_crypto_ops.py:133:    new = db_key.peek_key()
tests\test_db_crypto_ops.py:149:    with dbmod.connect(dest, key=db_key.peek_key()) as conn:
tests\test_recovery.py:63:    from muneem import db_key
tests\test_recovery.py:70:    original = db_key.peek_key()
```

```
tests\test_recovery.py:80:    assert db_key.peek_key() is None
tests\test_recovery.py:84:    assert db_key.peek_key() == original  # key restored to the
keychain
tests\test_recovery.py:85:    with dbmod.connect(led, key=db_key.peek_key()) as conn:
```

6. user (2026-06-10T16:22:56.468Z)

```
360             part = (row["particulars"] or "")[:50]
361             typer.echo(f"[{row['account_number']}] {row['txn_date']} : {part:<50} |
{amount:>10}")
362
363
364     @db_app.command("status")
365     def db_status(
366         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
367     ) -> None:
368         """Show the ledger's encryption state (no statement data is read or printed)."""
369         from muneem import db as dbmod
370         from muneem.db_key import peek_key
371
372         db_path = db or load_config().db_path
373         key_state = "present in keychain" if peek_key() else "not yet minted"
374         if not Path(db_path).exists():
375             typer.echo(f"ledger : {db_path} (does not exist yet)")
376             typer.echo(f"key    : {key_state}")
377             return
378         if dbmod.is_plaintext_sqlite(db_path):
379             state = "PLAINTEXT ? run 'muneem db encrypt' to harden it"
380         else:
381             state = "encrypted (SQLCipher)"
382         typer.echo(f"ledger : {db_path}")
383         typer.echo(f"state  : {state}")
384         typer.echo(f"key    : {key_state}")
385
386
387     @db_app.command("encrypt")
388     def db_encrypt(
389         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
390     ) -> None:
391         """Migrate an existing *plaintext* ledger to an encrypted one (SQLCipher), in
place."""
392         from muneem import db as dbmod
393         from muneem.db_key import get_or_create_key
394
395         db_path = db or load_config().db_path
396         if not Path(db_path).exists():
397             typer.echo(
398                 f"nothing to encrypt: {db_path} does not exist (a new ledger is born
encrypted)."
399             )
400             return
401         if not dbmod.is_plaintext_sqlite(db_path):
402             typer.echo(
403                 f"{Path(db_path).name} is already encrypted (or not a SQLite file); nothing
to do."
404             )
405             return
406         n = dbmod.encrypt_database(db_path, get_or_create_key())
407         typer.echo(
408             f"encrypted {Path(db_path).name}: {n} table(s) migrated. A plaintext backup
remains at "
409             f"{Path(db_path).name}.plaintext.bak ? verify, then securely delete it."
410         )
```

```
411
412
413     def _require_encrypted_ledger(db_path: Path) -> str:
414         """Return the ledger's keychain key, or exit with a clear message if it can't be
used."""
415         from muneem import db as dbmod
416         from muneem.db_key import peek_key
417
418         if not Path(db_path).exists() or dbmod.is_plaintext_sqlite(db_path):
419             typer.echo("no encrypted ledger here (run 'muneem db encrypt' first).",
err=True)
420             raise typer.Exit(1)
421         key = peek_key()
422         if not key:
423             typer.echo("no encryption key in the keychain for this ledger.", err=True)
424             raise typer.Exit(1)
425         return key
426
427
428     @db_app.command("rotate-key")
429     def db_rotate_key(
430         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
431     ) -> None:
432         """Rotate the ledger's encryption key (SQLCipher rekey) and update the keychain."""
433         from muneem import db as dbmod
434         from muneem import db_key
435
436         db_path = db or load_config().db_path
437         current = _require_encrypted_ledger(db_path)
438         fresh = db_key.new_key()
439         # Keychain-first with rollback: if the file rekey fails, the keychain still matches
the file.
440         db_key.set_key(fresh)
441         try:
442             dbmod.rekey_database(db_path, current, fresh)
443         except dbmod.EncryptedDatabaseError as exc:
444             db_key.set_key(current)  # roll back; the file is still under the old key
445             typer.echo(str(exc), err=True)
446             raise typer.Exit(1) from exc
447         typer.echo(f"rotated the encryption key for {Path(db_path).name}.")
448
449
450     @db_app.command("backup")
451     def db_backup(
452         dest: Path = typer.Argument(..., help="Destination path for the encrypted backup
copy."),
453         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
454     ) -> None:
455         """Write an encrypted online-backup copy of the ledger (consistent even with WAL
active)."""
456         from muneem import db as dbmod
457
458         db_path = db or load_config().db_path
459         key = _require_encrypted_ledger(db_path)
460         try:
461             dbmod.backup_database(db_path, dest, key)
462         except dbmod.EncryptedDatabaseError as exc:
463             typer.echo(str(exc), err=True)
464             raise typer.Exit(1) from exc
465         typer.echo(f"backed up {Path(db_path).name} to {dest} (encrypted).")
466
467
468     @db_app.command("setup-recovery")
```

```
    def db_setup_recovery(
        out: Path | None = typer.Option(
            None, help="Recovery file path (default: <data dir>/muneem-recovery.json)."
        ),
    ) -> None:
        """Create a passphrase-wrapped recovery file for the ledger's encryption key.

        Lets you restore the key (and thus the ledger + backups) if the OS keychain is ever
lost. The
        file holds only the *wrapped* key ? useless without the passphrase.
        """
        import getpass

        from muneem import recovery
        from muneem.db_key import peek_key

        key = peek_key()
        if not key:
            typer.echo("no encryption key in the keychain yet (create the ledger first).",
err=True)
            raise typer.Exit(1)
        out_path = out or (load_config().data_dir / "muneem-recovery.json")
        passphrase = getpass.getpass("New recovery passphrase: ")
        if passphrase != getpass.getpass("Confirm passphrase: "):
            typer.echo("passphrases did not match.", err=True)
            raise typer.Exit(1)
        try:
            recovery.write_recovery_file(out_path, key, passphrase)
        except recovery.RecoveryError as exc:
            typer.echo(str(exc), err=True)
            raise typer.Exit(1) from exc
        typer.echo(
            f"wrote recovery file to {out_path}. Store it OFF this machine (password manager
/ "
            "separate drive) and remember the passphrase ? you need BOTH to recover. Without
them, a "
            "lost keychain means the ledger and its backups are unrecoverable."
        )


    @db_app.command("recover")
    def db_recover(
        file: Path = typer.Argument(..., exists=True, dir_okay=False, help="Recovery file
path."),
        db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
    ) -> None:
        """Restore the ledger's encryption key from a recovery file + passphrase (into the
keychain)."""
        import getpass

        from muneem import db as dbmod
        from muneem import db_key, recovery

        passphrase = getpass.getpass("Recovery passphrase: ")
        try:
            key = recovery.read_recovery_file(file, passphrase)
        except recovery.RecoveryError as exc:
            typer.echo(str(exc), err=True)
            raise typer.Exit(1) from exc
        db_path = db or load_config().db_path
        # If the ledger is present, verify the recovered key actually opens it before
committing it.
        if Path(db_path).exists() and not dbmod.is_plaintext_sqlite(db_path):
            try:
```

```
526                 with dbmod.connect(db_path, key=key) as conn:
527                     conn.execute("SELECT 1")
528             except dbmod.EncryptedDatabaseError as exc:
529                 typer.echo("recovered a key, but it does not open the current ledger.",
err=True)
530                 raise typer.Exit(1) from exc
531         db_key.set_key(key)
532         typer.echo("recovered the encryption key into the keychain.")
533
534
535     if __name__ == "__main__":
536         app()
537
```

7. user (2026-06-10T16:23:10.527Z)

```
.venv\Lib\site-packages\keyring\backends\Windows.py
```

8. user (2026-06-10T16:23:11.236Z)

```
tests\test_secrets.py:9:from keyring.errors import KeyringError
tests\test_secrets.py:12:from muneem.errors import KeychainError
tests\test_secrets.py:22:        raise KeyringError("no backend")
tests\test_secrets.py:36:        raise KeyringError("locked")
tests\test_secrets.py:39:    with pytest.raises(KeychainError) as ei:
tests\test_secrets.py:48:        raise KeyringError("nope")
```

9. user (2026-06-10T16:23:15.257Z)

```
1       from __future__ import annotations
2
3       import logging
4
5       from jaraco.context import ExceptionTrap
6
7       from ..backend import KeyringBackend
8       from ..compat import properties
9       from ..credentials import SimpleCredential
10      from ..errors import PasswordDeleteError
11
12      with ExceptionTrap() as missing_deps:
13          try:
14              # prefer pywin32-ctypes
15              from win32ctypes.pywin32 import pywintypes, win32cred
16
17              # force demand import to raise ImportError
18              win32cred.__name__  # noqa: B018
19          except ImportError:
20              # fallback to pywin32
21              import pywintypes
22              import win32cred
23
24              # force demand import to raise ImportError
25              win32cred.__name__  # noqa: B018
26
27      log = logging.getLogger(__name__)
28
29
30      class Persistence:
31          def __get__(self, keyring, type=None):
32              return getattr(keyring, '_persist', win32cred.CRED_PERSIST_ENTERPRISE)
33
34          def __set__(self, keyring, value):
35              """
```

```
            Set the persistence value on the Keyring. Value may be
            one of the win32cred.CRED_PERSIST_* constants or a
            string representing one of those constants. For example,
            'local machine' or 'session'.
            """
            if isinstance(value, str):
                attr = 'CRED_PERSIST_' + value.replace(' ', '_').upper()
                value = getattr(win32cred, attr)
            keyring._persist = value


    class DecodingCredential(dict):
        @property
        def value(self):
            """
            Attempt to decode the credential blob as UTF-16 then UTF-8.
            """
            cred = self['CredentialBlob']
            try:
                return cred.decode('utf-16')
            except UnicodeDecodeError:
                decoded_cred_utf8 = cred.decode('utf-8')
                log.warning(
                    "Retrieved a UTF-8 encoded credential. Please be aware that "
                    "this library only writes credentials in UTF-16."
                )
                return decoded_cred_utf8


    class WinVaultKeyring(KeyringBackend):
        """
        WinVaultKeyring stores encrypted passwords using the Windows Credential
        Manager.

        Requires pywin32

        This backend does some gymnastics to simulate multi-user support,
        which WinVault doesn't support natively. See
        https://github.com/jaraco/keyring/issues/47#issuecomment-75763152
        for details on the implementation, but here's the gist:

        Passwords are stored under the service name unless there is a collision
        (another password with the same service name but different user name),
        in which case the previous password is moved into a compound name:
        {username}@{service}
        """

        persist = Persistence()

        @properties.classproperty
        def priority(cls) -> float:
            """
            If available, the preferred backend on Windows.
            """
            if missing_deps:
                raise RuntimeError("Requires Windows and pywin32")
            return 5

        @staticmethod
        def _compound_name(username, service):
            return f'{username}@{service}'

        def get_password(self, service, username):
            res = self._resolve_credential(service, username)
            return res and res.value
```

```
101
102         def _resolve_credential(
103             self, service: str, username: str | None
104         ) -> DecodingCredential | None:
105             # first attempt to get the password under the service name
106             res = self._read_credential(service)
107             if not res or username and res['UserName'] != username:
108                 # It wasn't found so attempt to get it with the compound name
109                 res = self._read_credential(self._compound_name(username, service))
110             return res
111
112         def _read_credential(self, target):
113             try:
114                 res = win32cred.CredRead(
115                     Type=win32cred.CRED_TYPE_GENERIC, TargetName=target
116                 )
117             except pywintypes.error as e:
118                 if e.winerror == 1168 and e.funcname == 'CredRead':  # not found
119                     return None
120                 raise
121             return DecodingCredential(res)
122
123         def set_password(self, service, username, password):
124             existing_pw = self._read_credential(service)
125             if existing_pw:
126                 # resave the existing password using a compound target
127                 existing_username = existing_pw['UserName']
128                 target = self._compound_name(existing_username, service)
129                 self._set_password(
130                     target,
131                     existing_username,
132                     existing_pw.value,
133                 )
134             self._set_password(service, username, str(password))
135
136         def _set_password(self, target, username, password):
137             credential = dict(
138                 Type=win32cred.CRED_TYPE_GENERIC,
139                 TargetName=target,
140                 UserName=username,
141                 CredentialBlob=password,
142                 Comment="Stored using python-keyring",
143                 Persist=self.persist,
144             )
145             win32cred.CredWrite(credential, 0)
146
147         def delete_password(self, service, username):
148             compound = self._compound_name(username, service)
149             deleted = False
150             for target in service, compound:
151                 existing_pw = self._read_credential(target)
152                 if existing_pw and existing_pw['UserName'] == username:
153                     deleted = True
154                     self._delete_password(target)
155             if not deleted:
156                 raise PasswordDeleteError(service)
157
158         def _delete_password(self, target):
159             try:
160                 win32cred.CredDelete(Type=win32cred.CRED_TYPE_GENERIC, TargetName=target)
161             except pywintypes.error as e:
162                 if e.winerror == 1168 and e.funcname == 'CredDelete':  # not found
163                     return
164                 raise
165
```

```
166         def get_credential(self, service, username):
167             res = self._resolve_credential(service, username)
168             return res and SimpleCredential(res['UserName'], res.value)
169
```

10. user (2026-06-10T16:23:15.791Z)

```
1       """The OS-keychain wrappers: graceful read, attributed (value-free) write, swallowed
delete.
2
3       The autouse conftest fixture swaps ``muneem.secrets`` for an in-memory store, so we
capture the
4       *real* wrapper functions at import time and drive them by patching the underlying
``keyring`` lib.
5       """
6
7       import keyring
8       import pytest
9       from keyring.errors import KeyringError
10
11      import muneem.secrets as secretsmod
12      from muneem.errors import KeychainError
13
14      # Captured before the autouse keychain fixture rebinds the module attributes per-test.
15      _real_get = secretsmod.get_secret
16      _real_set = secretsmod.set_secret
17      _real_delete = secretsmod.delete_secret
18
19
20      def test_get_secret_returns_none_when_keyring_unavailable(monkeypatch):
21          def boom(service, name):
22              raise KeyringError("no backend")
23
24          monkeypatch.setattr(keyring, "get_password", boom)
25          assert _real_get("anything") is None  # graceful: absence, not a crash
26
27
28      def test_get_secret_returns_the_value(monkeypatch):
29          monkeypatch.setattr(keyring, "get_password", lambda s, n: "v" if n == "k" else None)
30          assert _real_get("k") == "v"
31          assert _real_get("missing") is None
32
33
34      def test_set_secret_raises_keychain_error_with_name_but_not_value(monkeypatch):
35          def boom(service, name, value):
36              raise KeyringError("locked")
37
38          monkeypatch.setattr(keyring, "set_password", boom)
39          with pytest.raises(KeychainError) as ei:
40              _real_set("db-key", "super-secret-value")
41          msg = str(ei.value)
42          assert "db-key" in msg  # the name is safe to surface
43          assert "super-secret-value" not in msg  # the value must NEVER appear
44
45
46      def test_delete_secret_swallows_keyring_errors(monkeypatch):
47          def boom(service, name):
48              raise KeyringError("nope")
49
50          monkeypatch.setattr(keyring, "delete_password", boom)
51          _real_delete("k")  # must not raise
52
```

11. assistant (2026-06-10T16:23:53.505Z)

Let me verify whether `keyring.get_password` wraps backend exceptions into `KeyringError`, since
on Windows the backend raises `pywintypes.error` for transient CredRead failures.

12. user (2026-06-10T16:23:54.489Z)

```
    """
    Core API functions and initialization routines.
    """

    from __future__ import annotations

    import configparser
    import logging
    import os
    import sys
    import typing

    from . import backend, credentials
    from .backends import fail
    from .util import platform_ as platform

    LimitCallable = typing.Callable[[backend.KeyringBackend], bool]

    log = logging.getLogger(__name__)

    _keyring_backend = None


    def set_keyring(keyring: backend.KeyringBackend) -> None:
        """Set current keyring backend."""
        global _keyring_backend
        if not isinstance(keyring, backend.KeyringBackend):
            raise TypeError("The keyring must be an instance of KeyringBackend")
        _keyring_backend = keyring


    def get_keyring() -> backend.KeyringBackend:
        """Get current keyring backend."""
        if _keyring_backend is None:
            init_backend()
        return typing.cast(backend.KeyringBackend, _keyring_backend)


    def disable() -> None:
        """
        Configure the null keyring as the default.

        >>> fs = getfixture('fs')
        >>> disable()
        >>> disable()
        Traceback (most recent call last):
        ...
        RuntimeError: Refusing to overwrite...
        """
        root = platform.config_root()
        try:
            os.makedirs(root)
        except OSError:
            pass
        filename = os.path.join(root, 'keyringrc.cfg')
        if os.path.exists(filename):
            msg = f"Refusing to overwrite {filename}"
            raise RuntimeError(msg)
        with open(filename, 'w', encoding='utf-8') as file:
            file.write('[backend]\ndefault-keyring=keyring.backends.null.Keyring')
```

```


    def get_password(service_name: str, username: str) -> str | None:
        """Get password from the specified service."""
        return get_keyring().get_password(service_name, username)


    def set_password(service_name: str, username: str, password: str) -> None:
        """Set password for the user in the specified service."""
        get_keyring().set_password(service_name, username, password)


    def delete_password(service_name: str, username: str) -> None:
        """Delete the password for the user in the specified service."""
        get_keyring().delete_password(service_name, username)


    def get_credential(
        service_name: str, username: str | None
    ) -> credentials.Credential | None:
        """Get a Credential for the specified service."""
        return get_keyring().get_credential(service_name, username)


    def recommended(backend) -> bool:
        return backend.priority >= 1


    def init_backend(limit: LimitCallable | None = None):
        """
        Load a detected backend.
        """
        set_keyring(_detect_backend(limit))


    def _detect_backend(limit: LimitCallable | None = None):
        """
        Return a keyring specified in the config file or infer the best available.

        Limit, if supplied, should be a callable taking a backend and returning
        True if that backend should be included for consideration.
        """

        # save the limit for the chainer to honor
        backend._limit = limit
        return (
            load_env()
            or load_config()
            or max(
                # all keyrings passing the limit filter
                filter(limit, backend.get_all_keyring()),
                default=fail.Keyring(),
                key=backend.by_priority,
            )
        )


    def _load_keyring_class(keyring_name: str) -> type[backend.KeyringBackend]:
        """
        Load the keyring class indicated by name.

        These popular names are tested to ensure their presence.

        >>> popular_names = [
        ...      'keyring.backends.Windows.WinVaultKeyring',
```

```
        ...      'keyring.backends.macOS.Keyring',
        ...      'keyring.backends.kwallet.DBusKeyring',
        ...      'keyring.backends.SecretService.Keyring',
        ...  ]
        >>> list(map(_load_keyring_class, popular_names))
        [...]
        """
        module_name, sep, class_name = keyring_name.rpartition('.')
        __import__(module_name)
        module = sys.modules[module_name]
        return getattr(module, class_name)


    def load_keyring(keyring_name: str) -> backend.KeyringBackend:
        """
        Load the specified keyring by name (a fully-qualified name to the
        keyring, such as 'keyring.backends.file.PlaintextKeyring')
        """
        class_ = _load_keyring_class(keyring_name)
        # invoke the priority to ensure it is viable, or raise a RuntimeError
        class_.priority  # noqa: B018
        return class_()


    def load_env() -> backend.KeyringBackend | None:
        """Load a keyring configured in the environment variable."""
        try:
            return load_keyring(os.environ['PYTHON_KEYRING_BACKEND'])
        except KeyError:
            return None


    def _config_path():
        return platform.config_root() / 'keyringrc.cfg'


    def _ensure_path(path):
        if not path.exists():
            raise FileNotFoundError(path)
        return path


    def load_config() -> backend.KeyringBackend | None:
        """Load a keyring using the config file in the config root."""

        config = configparser.RawConfigParser()
        try:
            config.read(_ensure_path(_config_path()), encoding='utf-8')
        except FileNotFoundError:
            return None
        _load_keyring_path(config)

        # load the keyring class name, and then load this keyring
        try:
            if config.has_section("backend"):
                keyring_name = config.get("backend", "default-keyring").strip()
            else:
                return None

        except (configparser.NoOptionError, ImportError):
            logger = logging.getLogger('keyring')
            logger.warning(
                "Keyring config file contains incorrect values.\n"
                + f"Config file: {_config_path()}"
            )
```

```
            return None

        return load_keyring(keyring_name)


    def _load_keyring_path(config: configparser.RawConfigParser) -> None:
        "load the keyring-path option (if present)"
        try:
            path = config.get("backend", "keyring-path").strip()
            sys.path.insert(0, os.path.expanduser(path))
        except (configparser.NoOptionError, configparser.NoSectionError):
            pass

```

13. user (2026-06-10T16:23:54.728Z)

```
    import sys
    import warnings


    class KeyringError(Exception):
        """Base class for exceptions in keyring"""


    class PasswordSetError(KeyringError):
        """Raised when the password can't be set."""


    class PasswordDeleteError(KeyringError):
        """Raised when the password can't be deleted."""


    class InitError(KeyringError):
        """Raised when the keyring could not be initialised"""


    class KeyringLocked(KeyringError):
        """Raised when the keyring failed unlocking"""


    class NoKeyringError(KeyringError, RuntimeError):
        """Raised when there is no keyring backend"""


    class ExceptionRaisedContext:
        """
        An exception-trapping context that indicates whether an exception was
        raised.
        """

        def __init__(self, ExpectedException=Exception):
            warnings.warn(
                "ExceptionRaisedContext is deprecated; use `jaraco.context.ExceptionTrap`",
                DeprecationWarning,
                stacklevel=2,
            )
            self.ExpectedException = ExpectedException
            self.exc_info = None

        def __enter__(self):
            self.exc_info = object.__new__(ExceptionInfo)
            return self.exc_info

        def __exit__(self, *exc_info):
            self.exc_info.__init__(*exc_info)
```

```
            return self.exc_info.type and issubclass(
                self.exc_info.type, self.ExpectedException
            )


    class ExceptionInfo:
        def __init__(self, *info):
            if not info:
                info = sys.exc_info()
            self.type, self.value, _ = info

        def __bool__(self):
            """
            Return True if an exception occurred
            """
            return bool(self.type)

        __nonzero__ = __bool__

```

14. user (2026-06-10T16:24:44.994Z)

Structured output provided successfully

15. assistant (2026-06-10T16:24:49.394Z)

Verification complete.